* Count Pair's "ag" -:

$i<j$ && $s[i] = 'a'$ and $s[j] = 'g'$

Example -:

	0	1	2	3	4	5	6	7
s =	b	a	a	g	d	c	a	g

(1, 3), (1, 7), (2, 3), (2, 7), (6, 7)

ans = 5

Idea -:

For each a, we are counting the no of g on it's right side.

c -> count of g's

ans -> count of 'ag' pairs

c = 0, ans = 0   g -> c++   ans -> ans += c

TC: O(N)

SC: O(1)

* Carry Forward:

We carry a certain value to be updated & used in the same loop.

* Leaders in an Array:

Example :

arr[8] = 15 -1 7 2 5 4 2 3

← ans=5

Idea:

Iterate & check for every element if it is a leaders

↳ For each arr[i]) max on it's right side

max[i+1 N-1]

For each element , we are computing the max on right side.

* Observation & optimisations:

Iterate right to left carry forward the max value.

maxVal - max of array so far
ans - final ans / count of leaders

TC: O(N)
SC: O(1)

* Subarray Basics:

1. Continuous part of an array is called Subarray.
2. A Single element is a subarray.
3. Entire array is a subarray.
4. Empty cannot be a subarray.

Example
a[ ] = -3 4 6 2 8 7 14 9 21

* Closest Min Max:

Example: [3 6]

arr[ ] = 1 2 3 1 3 4 6 4 6 3

0 1 2 3 4 5 6 7 8 9

length: 4

* observation-:

1) In final subarray min & max should be present at corners.

2) If you have min values, search for the first max on the right

3) If you have max value, search for the first min on the right

* optimisation-:

For each min -> first max index on right

For each max -> first min index on right

Iterat R To L

Carry forward maxIndex & minIndex

Code-:

1) Iterate & get minV & maxV

2) if minV == maxV return 1

3) Initialise ans = N, minIndex = -1, maxIndex = -1

4) Iterate R to L

TC: $O(N)$

SC: $O(1)$